

Lista de Exercícios de Fixação

1. Explique a diferença entre uma licença de software proprietária e uma licença open source. Dê exemplos de cada uma.

- Licença Proprietária: o código-fonte é fechado e pertence exclusivamente ao desenvolvedor ou empresa. O usuário paga pelo direito de uso, mas não pode modificar, redistribuir ou inspecionar o código. Exemplos: Microsoft Windows, Adobe Photoshop, AutoCAD.
- Licença Open Source: o código-fonte é disponibilizado publicamente, permitindo que qualquer pessoa visualize, modifique e redistribua o software, dentro das condições da licença específica. Exemplos: Linux (GPL), WordPress (GPL), VS Code (MIT), React (MIT).

A principal diferença está na liberdade de acesso ao código e nas permissões de uso. Softwares open source tendem a evoluir mais rapidamente pela contribuição da comunidade, enquanto os proprietários garantem maior controle e monetização ao desenvolvedor.

2. Pesquise as principais características de uma licença GNU GPL e descreva como ela impacta a distribuição e modificação de software.

A GNU GPL (General Public License) é uma das licenças open source mais utilizadas. Suas principais características são:

- Copyleft forte: qualquer software derivado ou que incorpore código GPL deve, obrigatoriamente, ser distribuído também sob a licença GPL. Isso garante que as liberdades do software original se propaguem para todos os derivados.
- Liberdade de uso, estudo, modificação e distribuição: qualquer pessoa pode utilizar o software para qualquer finalidade, estudar como funciona, modificá-lo e redistribuir as modificações.

Impacto na distribuição: empresas que incorporam código GPL em produtos comerciais são obrigadas a tornar o código-fonte do produto inteiro disponível publicamente. Isso limita o uso em produtos proprietários e é a razão pela qual algumas empresas preferem licenças mais permissivas (como MIT ou Apache) para suas dependências.

3. Descreva o processo de registro de software no INPI no Brasil e explique como ele pode ser útil para um desenvolvedor.

O registro de software no INPI (Instituto Nacional da Propriedade Industrial) é regulamentado pela Lei 9.609/1998 (Lei de Software). O processo envolve:

- Preenchimento do formulário de requerimento no portal do INPI, com informações sobre o software (nome, descrição, linguagem de programação, data de criação).
- Depósito de trechos do código-fonte (geralmente as primeiras e últimas páginas), que ficam sob sigilo.
- Pagamento de taxa e aguardo do processo de análise.

Utilidade para o desenvolvedor: o registro cria um documento oficial com data certa que comprova a autoria e a anterioridade do software. Em caso de disputa legal

(plágio, uso indevido ou reivindicação de autoria por terceiros), o registro serve como prova jurídica da criação original.

4. Explique a diferença entre direitos autorais e patentes em relação ao software. Em que situação o desenvolvedor de software pode requerer uma patente?

- Direitos Autorais: protegem a expressão criativa do software, ou seja, o código-fonte como obra intelectual. São automáticos no momento da criação, sem necessidade de registro formal, e têm vigência longa (geralmente a vida do autor mais 70 anos). No Brasil, a Lei 9.609/1998 equipara o software a uma obra literária para fins de direitos autorais.
- Patentes: protegem invenções com aplicação industrial - um processo, mecanismo ou método técnico inovador. No Brasil, a legislação não permite patentear software puro (algoritmos e código), mas permite patentear a invenção implementada por software quando ela produz um efeito técnico concreto além da programação em si.

Situação para requerer patente: um desenvolvedor pode requerer uma patente quando seu software implementa um método técnico inovador com resultado industrial tangível - por exemplo, um algoritmo de compressão de dados que reduz o tamanho de arquivos de uma forma tecnicamente nova e não óbvia.

5. Simule a criação de um contrato de desenvolvimento de software entre um freelancer e uma empresa, detalhando os principais itens que devem constar no documento.

Contrato de Desenvolvimento de Software - Itens principais:

- Identificação das partes: nome completo, CPF/CNPJ e endereço do freelancer e da empresa contratante.
- Objeto do contrato: descrição detalhada do sistema a ser desenvolvido, incluindo escopo, funcionalidades e tecnologias utilizadas.
- Prazo de entrega: cronograma de entregas parciais (milestones) e data de conclusão final.
- Valor e forma de pagamento: valor total, parcelas (ex.: 30% no início, 40% na entrega do MVP, 30% na conclusão) e forma de pagamento.
- Propriedade intelectual: definir se os direitos sobre o software pertencem à empresa contratante após o pagamento integral ou se o desenvolvedor retém algum direito.
- Confidencialidade (NDA): cláusula que proíbe o freelancer de divulgar informações estratégicas da empresa durante e após o contrato.
- Manutenção e suporte: definir o período de garantia pós-entrega e as condições para suporte técnico adicional.
- Penalidades: multas por atraso de entrega ou não pagamento, e condições para rescisão contratual.

6. Descreva como um acordo de confidencialidade (NDA) pode proteger o código-fonte e outras informações sensíveis durante um projeto de software.

Um NDA (Non-Disclosure Agreement / Acordo de Não Divulgação) é um contrato legal que obriga as partes signatárias a manter em sigilo as informações

confidenciais compartilhadas durante um projeto. No contexto de software, ele protege:

- Código-fonte: impede que o desenvolvedor use o código em outros projetos ou o compartilhe com terceiros.
- Algoritmos e lógica de negócio proprietária: protege soluções técnicas inovadoras que representam vantagem competitiva da empresa.
- Dados de clientes e informações estratégicas: garante conformidade com a LGPD e evita vazamentos de informações sensíveis do negócio.

Em caso de violação do NDA, a parte prejudicada pode acionar judicialmente a outra parte e buscar indenização pelos danos causados, tornando o contrato um forte desincentivo à divulgação indevida.

7. Dê um exemplo de como o uso de software sem a devida licença pode gerar problemas legais para uma empresa.

Exemplo: uma empresa de médio porte utiliza 20 cópias de um software de design proprietário sem adquirir as licenças correspondentes (pirataria corporativa). O fabricante do software realiza uma auditoria (prática comum no mercado B2B) e detecta as instalações não licenciadas.

Consequências legais:

- Ação civil por violação de direitos autorais, com indenização que pode chegar a valores muito superiores ao custo das licenças originais.
- Multas administrativas e possível inquérito criminal, dependendo da legislação local.
- Danos à reputação da empresa, especialmente se o caso se tornar público.

A auditoria de softwares e a conformidade de licenças (SAM - Software Asset Management) são práticas essenciais para evitar esse tipo de exposição legal.

8. Explique como um desenvolvedor pode licenciar seu software sob uma licença proprietária e ainda permitir que outros desenvolvedores criem extensões ou plug-ins para seu sistema.

Esse modelo é chamado de 'plataforma proprietária com ecossistema aberto'. O desenvolvedor mantém o núcleo do software fechado e proprietário, mas disponibiliza uma API pública ou SDK (kit de desenvolvimento) que permite a terceiros criar extensões sem ter acesso ao código interno.

Como funciona na prática:

- O desenvolvedor publica uma documentação da API com os endpoints e funcionalidades disponíveis para integração.
- Define uma licença específica para o SDK que permite criar plugins, mas impede a engenharia reversa do núcleo.
- Pode criar uma marketplace de extensões (como a App Store da Apple ou a marketplace do VS Code) para monetizar o ecossistema.

Exemplos reais: o Shopify permite que desenvolvedores criem aplicativos para sua plataforma usando uma API pública, enquanto o código principal do Shopify permanece proprietário e fechado.

9. Pesquise um caso real de disputa legal envolvendo direitos autorais de software e descreva como os direitos autorais ou o registro de software foram utilizados no caso.

Caso: Oracle vs. Google (2010–2021).

A Oracle processou o Google alegando que o Android infringiu direitos autorais ao reproduzir 11.330 linhas de código da API Java, além da estrutura e organização das APIs do Java SE, que a Oracle adquiriu com a compra da Sun Microsystems em 2010.

Como os direitos autorais foram utilizados:

- A Oracle argumentou que a estrutura, sequência e organização das APIs Java eram obras protegidas por direitos autorais e que o Google as copiou sem licença.
- O Google argumentou que o uso era coberto pelo princípio de 'uso justo' (fair use), pois as APIs eram necessárias para a interoperabilidade.
- Em 2021, a Suprema Corte dos EUA decidiu a favor do Google, concluindo que o uso das APIs Java no Android constituía uso justo, encerrando uma das disputas de propriedade intelectual mais longas da indústria de tecnologia.

10. Descreva os benefícios de se utilizar uma licença open source em um projeto de software colaborativo. Que tipo de desafios isso pode apresentar para os desenvolvedores?

Benefícios:

- Colaboração global: desenvolvedores do mundo inteiro podem contribuir com melhorias, correção de bugs e novas funcionalidades, acelerando o desenvolvimento.
- Transparência e confiança: o código aberto pode ser auditado por qualquer pessoa, aumentando a confiança em termos de segurança e qualidade.
- Adoção mais rápida: a ausência de custo de licença reduz a barreira de adoção, especialmente em ambientes educacionais e startups.

Desafios:

- Monetização: transformar um projeto open source em negócio sustentável é complexo, exigindo modelos como suporte pago, versão enterprise ou doações.
- Gestão de contribuições: avaliar, revisar e integrar contribuições de dezenas de desenvolvedores externos demanda tempo e processos de governança claros.
- Compatibilidade de licenças: ao utilizar dependências open source com licenças diferentes (ex.: GPL e MIT), o desenvolvedor precisa garantir que as licenças são compatíveis entre si para evitar violações legais.